

EXPERIMENT – 7

Name: K ANUSRI

Roll No: 2520030432

Section: 8

Title

Observation of Memory Addresses and Process Memory Mapping in Linux

Aim

To observe the memory addresses of different variables and functions in a process and examine the process memory layout using `/proc/PID/maps`.

Theory

A process uses different memory regions during execution.

- **Code/Text:** Contains the program instructions and functions.
- **Global Memory:** Stores global variables.
- **Static Memory:** Stores static variables.
- **Heap:** Used for dynamically allocated memory using `malloc()`.
- **Stack:** Stores local variables and function-related data.
- **`/proc/PID/maps`:** Displays the memory regions mapped into a running process.

Algorithm

1. Declare a global variable.
2. Declare a static variable inside `main()`.
3. Declare a local variable on the stack.
4. Allocate memory dynamically using `malloc()`.
5. Display the addresses of the code, global, static, heap, and stack regions.
6. Display the process ID.
7. Keep the process running temporarily using `sleep()`.
8. Open another terminal.
9. Use `/proc/PID/maps` with the process ID to observe the memory mappings.
10. Release the dynamically allocated memory.

Program Code

```
k_anusri@DESKTOP-TM20BAQ: ~$  
k_anusri@DESKTOP-TM20BAQ: ~$ cat > exp7_memory.c <<'EOF'  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
  
int global_var = 10;  
  
void code_function()  
{  
}  
  
int main()  
{  
    static int static_var = 20;  
    int stack_var = 30;  
    int *heap_var = malloc(sizeof(int));  
  
    *heap_var = 40;  
  
    printf("PID: %d\n", getpid());  
    printf("Code    address   : %p\n", (void *)code_function);  
    printf("Global address : %p\n", (void *)&global_var);  
    printf("Static address : %p\n", (void *)&static_var);  
    printf("Heap    address   : %p\n", (void *)heap_var);  
    printf("Stack   address   : %p\n", (void *)&stack_var);  
  
    printf("\nProcess is running...\n");  
    sleep(20);  
  
    free(heap_var);  
  
    return 0;  
}  
EOF
```

Execution and Screenshots

Screenshot 1 – Compilation and Program Execution

```

k_anusri@DESKTOP-TM20BAQ:~$ gcc exp7_memory.c -o exp7_memory
./exp7_memory
PID: 732
Code    address   : 0x6516d13a1209
Global  address   : 0x6516d13a4010
Static  address   : 0x6516d13a4014
Heap    address   : 0x6516d4266010
Stack   address   : 0x7fff800aaa1c

Process is running...

```

Screenshot 2 – Process Memory Mapping
IN TERMINAL 2

```

k_anusri@DESKTOP-TM20BAQ:~$ cat /proc/732/maps
6516d13a0000-6516d13a1000 r--p 00000000 08:30 13632 /home/k_anusri/exp7_memory
6516d13a1000-6516d13a2000 r-xp 00001000 08:30 13632 /home/k_anusri/exp7_memory
6516d13a2000-6516d13a3000 r--p 00002000 08:30 13632 /home/k_anusri/exp7_memory
6516d13a3000-6516d13a4000 r--p 00002000 08:30 13632 /home/k_anusri/exp7_memory
6516d13a4000-6516d13a5000 rw-p 00003000 08:30 13632 /home/k_anusri/exp7_memory
6516d4266000-6516d4287000 rw-p 00000000 00:00 0 [heap]
720aa5600000-720aa5620000 r--p 00000000 08:30 13647 /usr/lib/x86_64-linux-gnu/libc.so.6
720aa5620000-720aa57c0000 r-xp 00020000 08:30 13647 /usr/lib/x86_64-linux-gnu/libc.so.6
720aa57c0000-720aa580e000 r--p 001c0000 08:30 13647 /usr/lib/x86_64-linux-gnu/libc.so.6
720aa580e000-720aa5812000 r--p 0020d000 08:30 13647 /usr/lib/x86_64-linux-gnu/libc.so.6
720aa5812000-720aa5814000 rw-p 00211000 08:30 13647 /usr/lib/x86_64-linux-gnu/libc.so.6
720aa5814000-720aa5821000 rw-p 00000000 00:00 0
720aa583b000-720aa583e000 rw-p 00000000 00:00 0
720aa5840000-720aa5846000 rw-p 00000000 00:00 0
720aa5846000-720aa584a000 r--p 00000000 00:00 0 [vvar]
720aa584a000-720aa584c000 r--p 00000000 00:00 0 [vvar_vclock]
720aa584c000-720aa584f000 r-xp 00000000 00:00 0 [vdso]
720aa584f000-720aa584f000 r--p 00000000 08:30 13644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
720aa584f000-720aa587e000 r-xp 00001000 08:30 13644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
720aa587e000-720aa5889000 r--p 00030000 08:30 13644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
720aa5889000-720aa588b000 r--p 0003b000 08:30 13644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
720aa588b000-720aa588c000 rw-p 0003d000 08:30 13644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
720aa588c000-720aa588d000 rw-p 00000000 00:00 0
7fff8008c000-7fff800ad000 rw-p 00000000 00:00 0 [stack]

```

Result

The memory addresses of the code, global, static, heap, and stack regions were successfully observed. The process memory mappings were successfully examined using /proc/PID/maps.